Module	Computer Fundamentals / COMP1027 (CSF) / Semester 1
Module Convenor(s)	Tissa Chandesa

Assessment Name	Coursework 2	**Weight**	35%

Description and Deliverable(s)

The coursework (details below) focuses on **Socket Programming**. It is an **INDIVIDUAL WORK.**

Preliminaries

- In this part of the coursework, a client-server implementation in C will be our focus. By now, you would have successfully created a client-server environment using *Visual Studio* IDE. If for some weird reason you have not done this yet – please refer to **laboratory sheet – Socket Programming** for more details on how to perform this.

Files to Download

1. Download the relevant file(s) from the "Coursework 2" area on Moodle.

Important Notes

1. Your program should be clearly structured and have proper comments.
2. I **strongly advised** that you code this **on your own** without the use of any AI chatbots.
3. **No External libraries** to be used (**unless approved beforehand**) and included within the submission. This is important to avoid compilation failure (and subsequently **loss of marks**). Below are **suggested** libraries you may use:
   - *stdio.h*
   - *netdb.h*
   - *netinet/in.h*
   - *unistd.h*
   - *fcntl.h*
   - *stdlib.h*
   - *string.h*
   - *sys/socket.h*
   - *sys/types.h*
   - *arpa/inet.h*
   - *ctype.h*
   - *time.h*

   Note: some of the above libraries are applicable to only MacOS/Linux users. For Windows users, you may use those that are applicable.
4. Please make sure to use the **correct argumentative channel** (e.g., **argv[0]**, **argv[1]**). The use of other argumentative channels will result in your codes being **unable to run**, subsequently **affecting your marks for that task**.

Task 1:

Modify the original server code so that its output *alphanumeric* message is in ***reverse*** order to the client's input message. Please note that all alphabets **MUST** be converted to ***uppercase*** at the server side. Your input (in the client side) can be a mixture of *upper* and *lowercase*.

SUBMISSION: The modified server should be saved as **Server1.c**.

Task 2:

Improve the server code (**from Server1.c**) to handle **multiple requests** (instead of one) from the same session. For each new connection, the server window should display:

1. The **IP address** of the client,
2. The **port number** targeted by the client's request.

For each message received, the server should **display the length** of the message. Your codes should also handle **errors** should the client send over unique characters as a message. An **error message** should be returned from the server to the client to inform the client of these errors and to key in an alphanumeric format message only.

Finally, the modified server code should close the connection and terminate if the client input the message: "**Exit Server**".

SUBMISSION: The modified server should be saved as **Server2.c**.

Task 3:

Modify the server code (**from Server2.c**) to handle specific **COMMANDS** sent from the client. A particular command, in this task, is "**Date**".

Type "**Date**" in the client, which will be sent to the server, the server should respond with the ***Current Day, Date and Time*** (in its *Current Time-Zone*, e.g., KL).

1. This should be presented as a single line, that is terminated with a carriage return (ASCII code 13) and line feed (ASCII code 10).
2. Example of output: *Wed Nov 30 17:00:54 2024 GMT + 8*

If any other characters (i.e., not command), the server will handle them with an error message as in task 2.

SUBMISSION: The modified server should be saved as **Server3.c**.

Task 4:

Modify the server code (**from Server3.c**) to handle additional commands sent from the client. A particular command, in this task, is "**Time**" and a code of a time-zone.

Typing "**Time**" in the client, which will be sent to the server, the server should respond with only the **Current Time** (in the **Current Time-Zone**, which is in "*Malaysia*" time).

Typing, for example, "**Time GMT**" in the client, which will be sent to the server, the server should respond with the Current Time (in the **GMT Time-Zone**). List of acceptable time-zones and the associated offset from GMT (in hours), is available in Table 1, below. *Please assume that the server is based in Malaysia, hence, you would need to modify the offsets in accordance with the current time-zone, so that the returned times are correct.*

If any other Time-Zone codes provided, the server should respond with an error message as in task 2 and 3.

Table 1. Time Zones

Time-zone Command	Offset from GMT (in hours)
PST	-8
MST	-7
CST	-6
EST	-5
GMT	0
CET	+1
MSK	+3
JST	+9
AEDT	+11

Expansion of the above-mentioned abbreviation can be found here.

SUBMISSION: The modified server should be saved as **Server4.c**.

Task 5:

Create two clients and one server that allows both clients to communicate through a server in real-time. Both clients should be able to communicate with infinite number of text (*e.g., your WhatsApp Chat Group*) where one client send text to the server and the other client sees the text from the first one on the screen.

Requirements:

1. Server:
   - Listen for and accept connections from two clients.
   - Forward messages between clients.
   - Handle multiple clients using threads.
2. Client:
   - Connect to the server.
   - Send and receive messages in real time.
   - Display received messages clearly.
   - Allow continuous messaging until the user decides to exit.
   - Implement commands like "**Exit Server**" to disconnect.

SUBMISSION: The modified server should be saved as **Server5.c**, along with two clients, named as: **FirstClient.c and SecondClient.c**

Final Submission Instructions

You are required to submit **ALL** the required tasks, as per the coursework instructions above (see the "**SUBMISSION**" line under each task).

Your comments within the code files will serve as your brief reporting on the tasks.

For **ALL tasks**, each student should submit his/her own files (*as listed* below). **We reserve the right to ask all/some students to explain any/all of your submitted work at any time. Failure to explain it properly could affect YOUR mark.**

Adherence to the files' naming rule (i.e., Server1.c **IS NOT THE SAME AS** *server1.c*). Changing the letter(s) of submitted file(s) will result in you **being awarded a 0%** on that corresponding task. So, please be careful!

The same applies to the given command instruction(s) for each task – i.e., Exit Server **IS NOT THE SAME AS** *exit server* and Date **IS NOT THE SAME AS** *DATE*. Changing these command instructions will result in you **being awarded a 0%** for that corresponding task. So, please be careful!

All students **MUST** submit on Moodle a **ZIP** archive file. **Any other archive file formats WILL result in a penalty of a 5% deduction of your overall mark.** Within your ZIP archive, it should contain all your individual worked files. Name that ZIP file as **CSF-CW2-XXXXXXXX.zip**, where the "XXXXXXXX" is the 8-digits of your student ID.

All the files should have the (**EXACT**) file names as detailed below:

Client.c
Server1.c
Server2.c
Server3.c
Server4.c

Server5.c
FirstClient.c
SecondClient.c
CW 2 Marking Sheet.docx

Please fill in the required details in **CW 2 Marking Sheet.docx** as this sheet will be handed back to you as your feedback for Coursework 2. Failure to fill in the required details **WILL result in a penalty of 15% deduction of your overall mark**. So, please be careful!

On Moodle, please click on the "**Coursework 2**" link within the "Coursework" section to perform **your** submission.

Release Date	Monday, 18th November 2024
Submission Date	Friday, 13th December 2024, by 11:59pm
Late Policy (University of Nottingham default will apply, if blank)	Work submitted after the deadline will be subject to a penalty of 5 marks (the standard 5% absolute) for each late **working day** out of the total 100 marks.
Feedback Mechanism and Date	Marks and written individual feedback will be returned via Moodle within the week commencing 20 January 2025.

Assessment Criteria

Assessment Breakdown:

Components	Marks Distribution
Task 1: Server1	
- Outputs in uppercase for alphabets ONLY; numerals remain the same	10
- Reverse order outputs	5
Task 2: Server2	
- Multiple requests handling	5
- Display of IP address	5
- Display the port number	5
- Display the message length	5
- Display error message	5
- Implement the command "Exit Server"	5
Task 3: Server3	
- Server3, handling the "Date" command (as in the CW sheet), successfully compile & run.	10
Task 4: Server4	
- Server4, handling the "Time" command (and its parameters, as per the CW sheet), successfully compile & run.	10
Task 5: Server5	

	- Server5, handling the incoming messages from two clients and sending one client's text to another and vice versa.	15
	- Successfully displays multiple messages from both clients whereby both can see the messages appearing on the server	15
	- Implement the command “Exit Server”	5